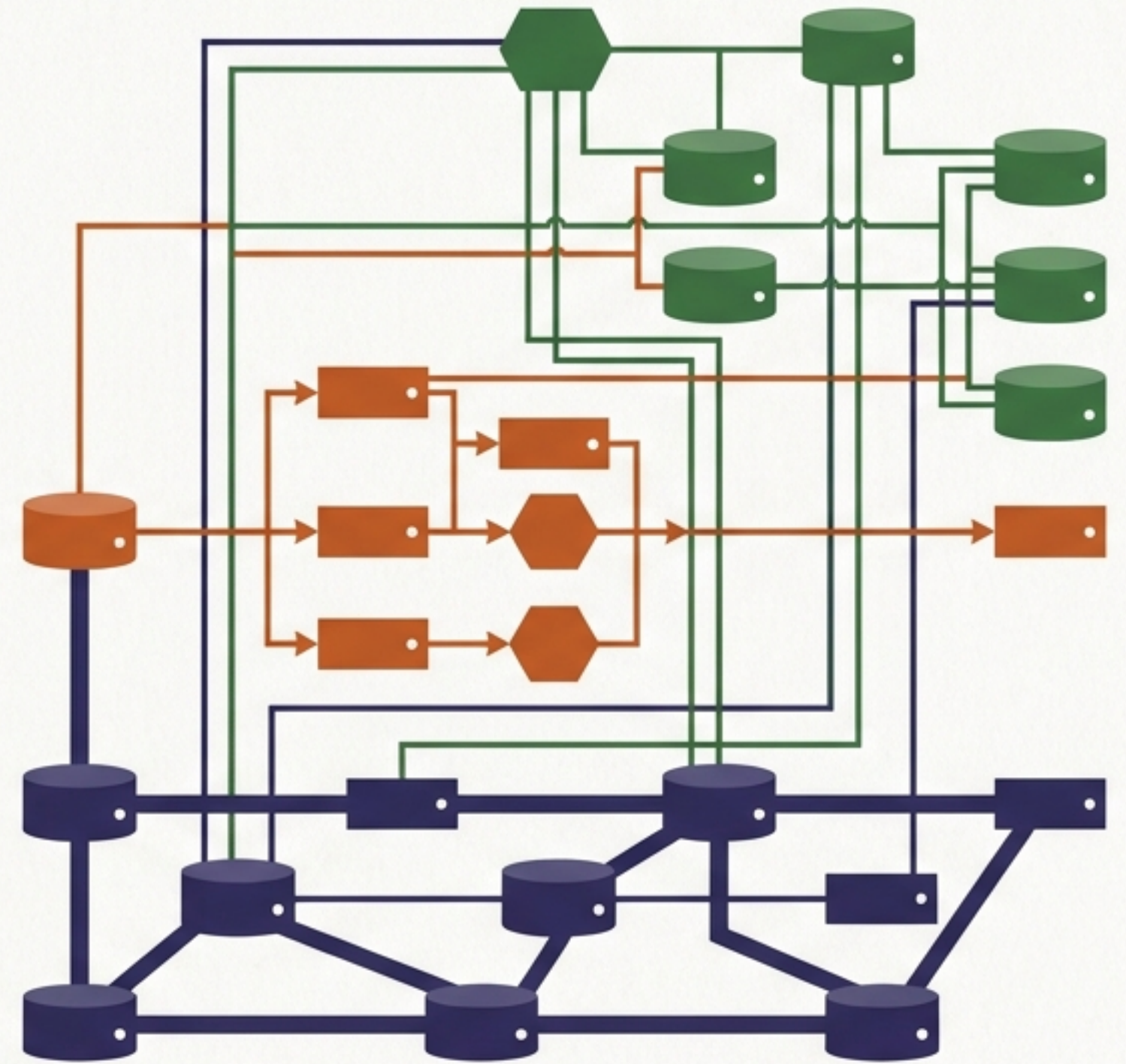


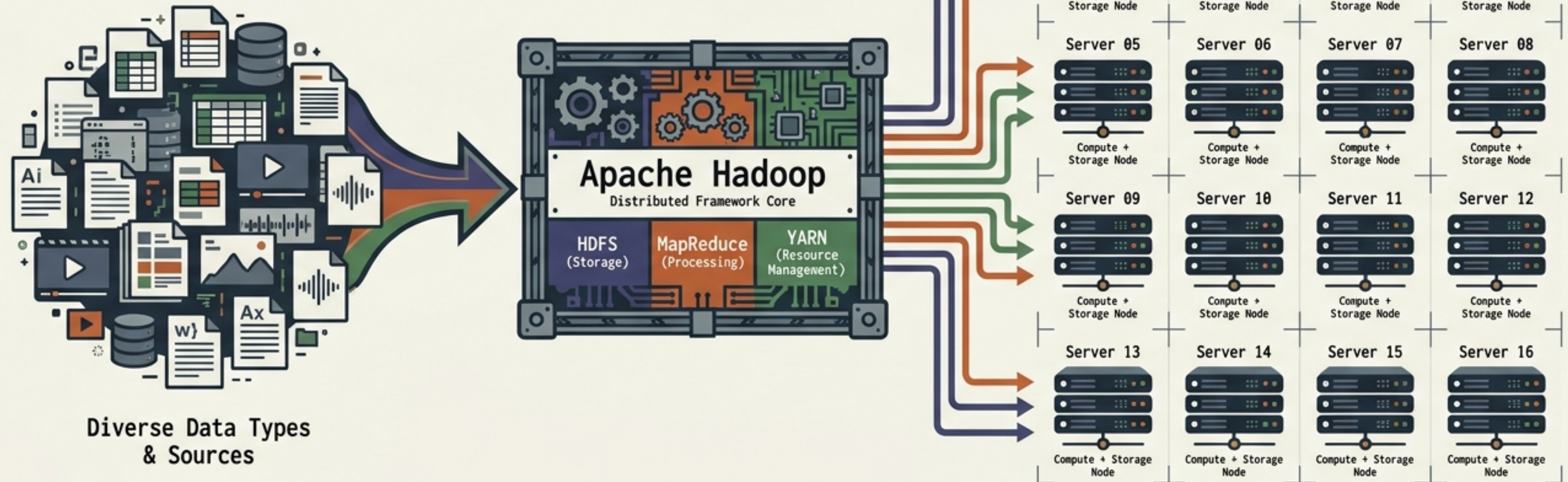
The Complete Blueprint to Hadoop & HDFS

Comprehensive study notes on distributed big data architecture, cluster management, and command-line operations.



Hadoop distributes data computation across massive server grids

Hadoop is an open-source framework designed to store and process enormous datasets in a distributed computing environment. It seamlessly scales from a single local machine to thousands of commodity servers, each offering local computation and storage capabilities.



Big data requires a paradigm shift from traditional databases

Traditional RDBMS

Technology Focus:
MySQL / Oracle

Processing Method:
Serial processing

Data Compatibility:
Strictly structured data (SQL tables)

Scalability:
Vertical, hardware-limited

Hadoop Framework

Technology Focus:
Apache Hadoop

Processing Method:
Parallel distributed processing

Data Compatibility:
Structured, Semi-structured (JSON, XML),
and Unstructured

Scalability:
Horizontal, limitless across commodity
hardware

Four pillars of the Hadoop architecture



Highly Scalable

Distributes massive datasets across thousands of nodes for rapid parallel processing.



Cost-Effective

100% open-source and runs entirely on standard, affordable commodity hardware.



Fault-Tolerant

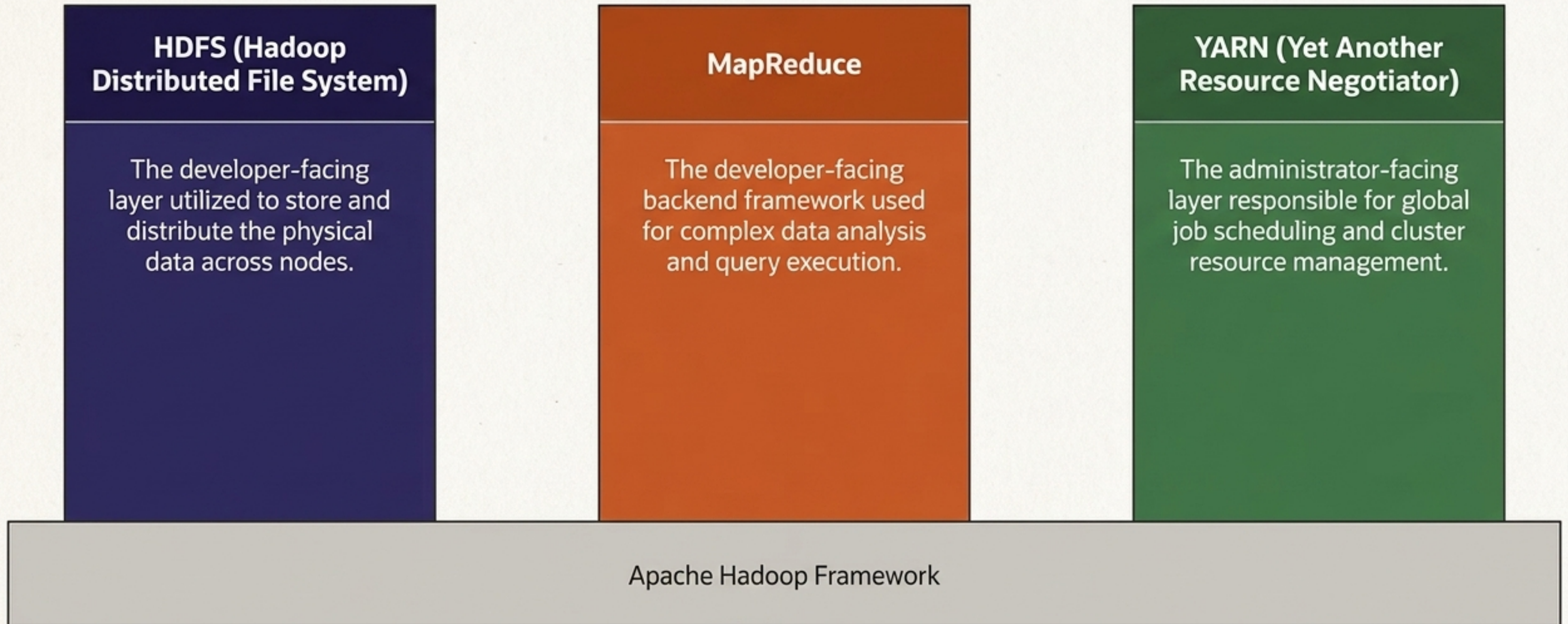
Automatically replicates data across multiple nodes. If a hardware failure occurs, data is instantly recoverable.



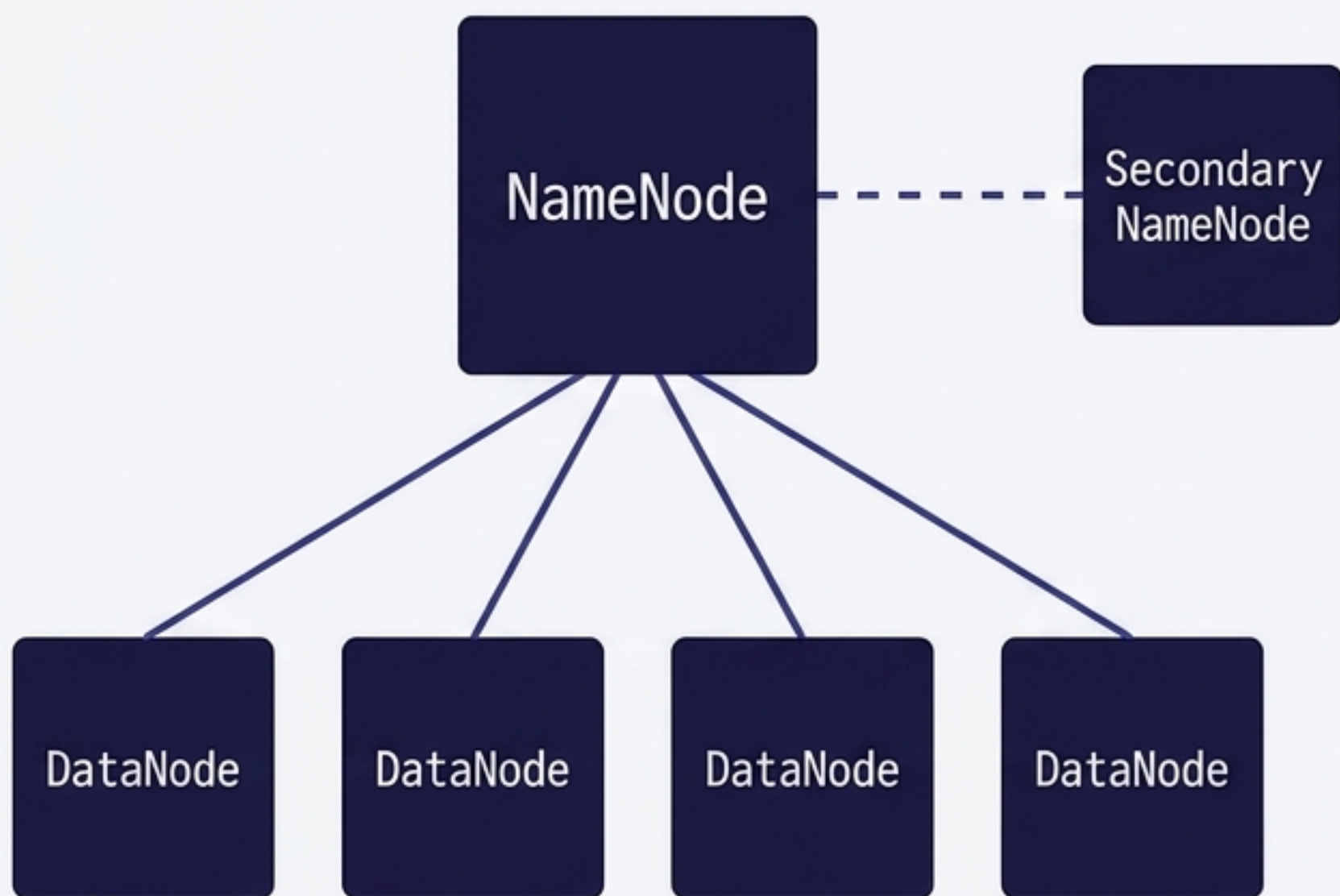
Highly Available

Master-slave backups ensure continuous operation without single points of failure.

The Hadoop ecosystem relies on three foundational layers



HDFS utilizes a strict Master-Slave topology



NameNode (The Master)

Maintains the critical metadata. It does not hold data, but tracks file names, permissions, and exact block locations across the cluster.

DataNode (The Slaves)

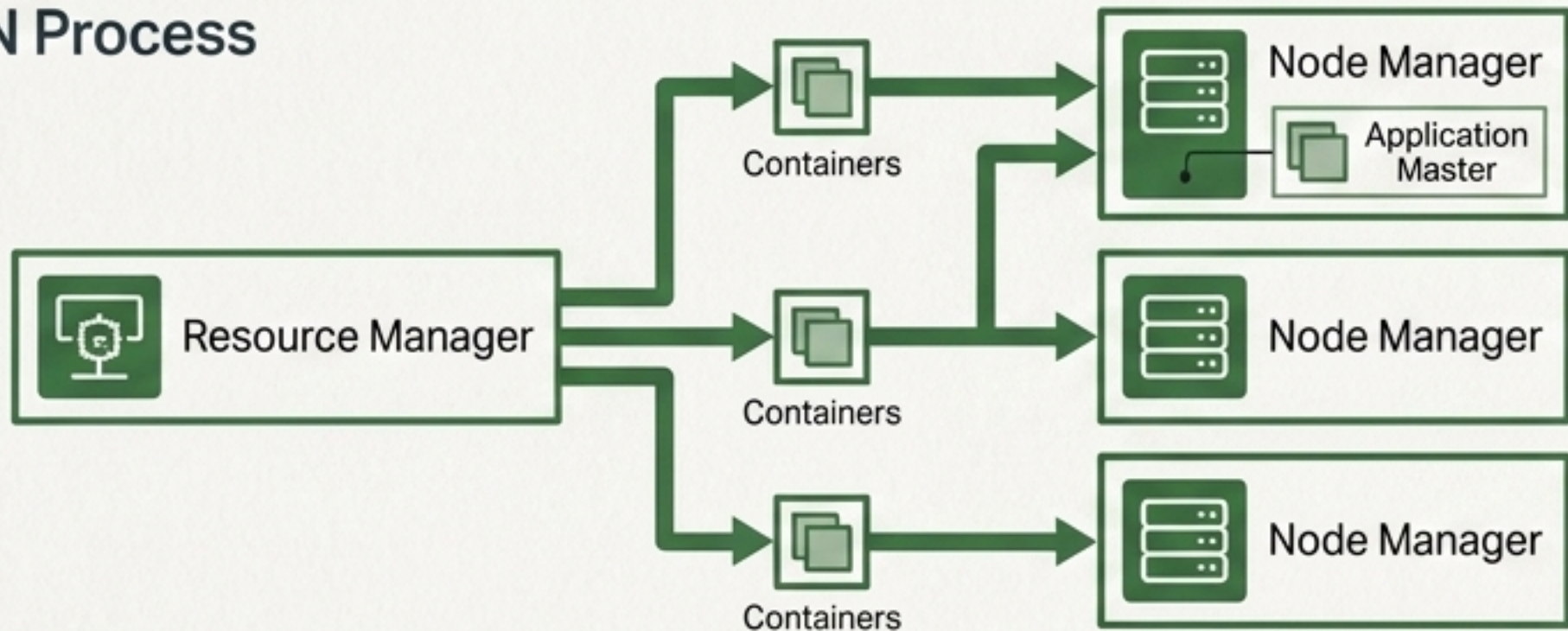
The physical storage units. Stores the actual data split into rigid 128 MB blocks.

Secondary NameNode (The Assistant)

Assists the Master by maintaining metadata checkpoints. Crucial distinction: It is NOT a backup for the NameNode.

YARN manages the hardware while MapReduce processes the logic

The YARN Process



Resource Manager
Allocates global cluster hardware.

Node Manager
Manages hardware on individual physical servers.

Application Master & Containers
Manages specific job execution within allocated CPU and Memory limits.

The MapReduce Process



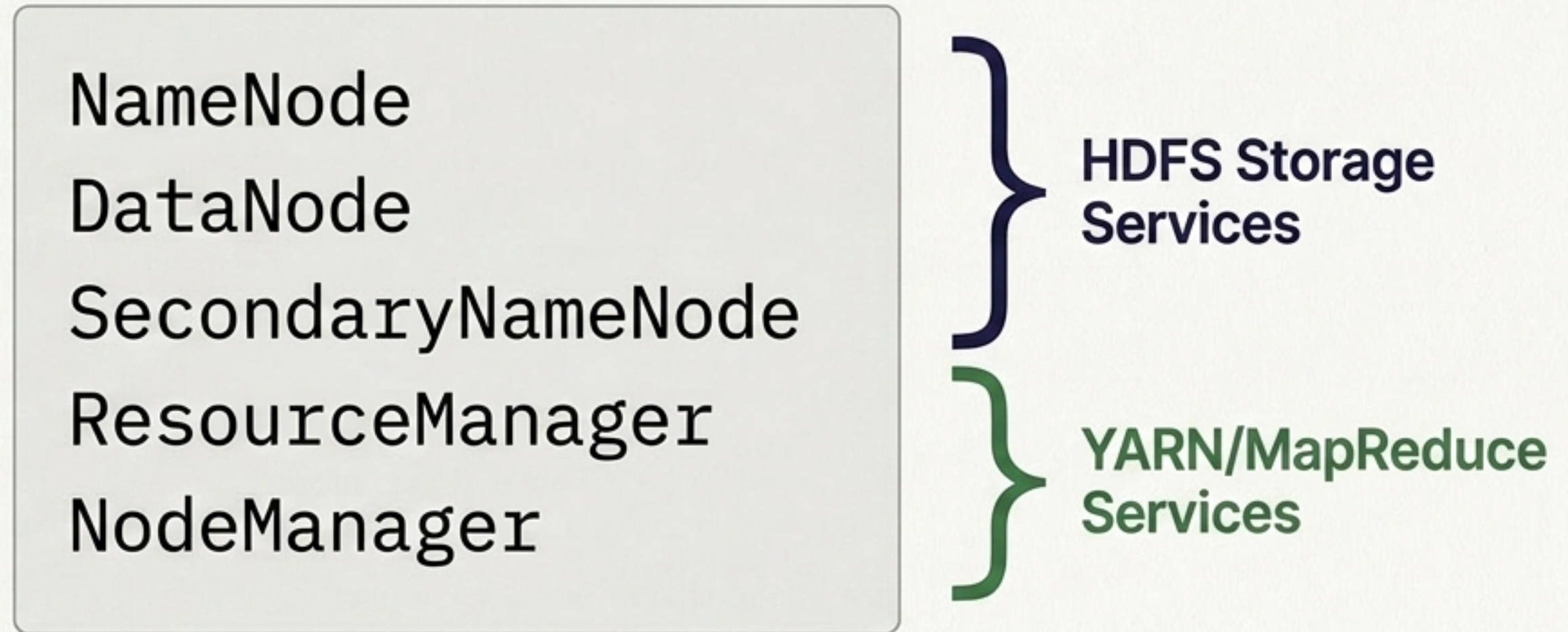
Map Phase
Ingests and processes the parallel input data.

Reduce Phase
Aggregates the mapped outputs into the final refined result.

System Diagnostics:

Verifying cluster health with 'jps'

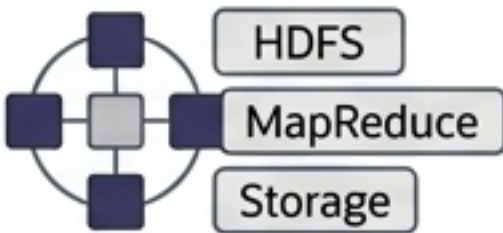
After starting the cluster (sbin/start-all.sh), running the Java Virtual Machine Process Status Tool (jps) must return five specific running services to confirm a successful boot.



Note: If any of these five are missing, the installation or boot sequence has failed.

Hadoop is the prerequisite foundation for modern cloud data engineering

The Foundation (Apache Hadoop)



The original distributed storage and processing framework.

The In-Memory Evolution (PySpark / Apache Spark)



Replaced MapReduce for faster, memory-based data analysis while often still relying on HDFS principles.

The Cloud Era (AWS Glue / Athena / Azure Data Factory)



Modern managed cloud services built directly on the architectural paradigms established by PySpark and Hadoop.

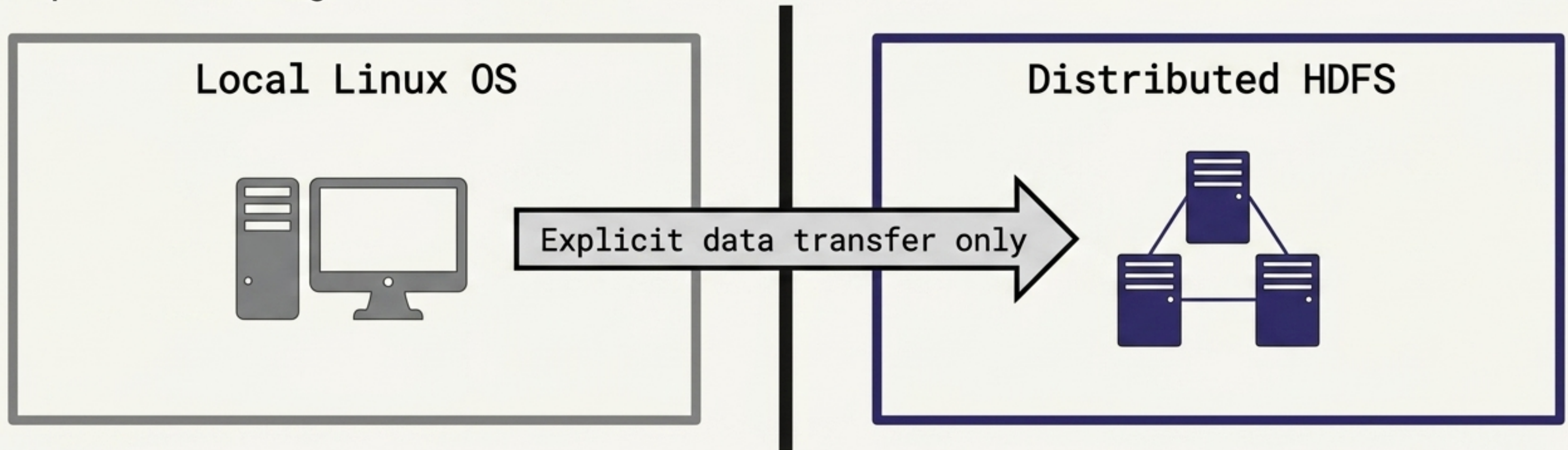


What about Kubernetes/Docker?

Container orchestration tools replicate host Operating Systems (Linux), whereas Hadoop strictly distributes data storage and computation.

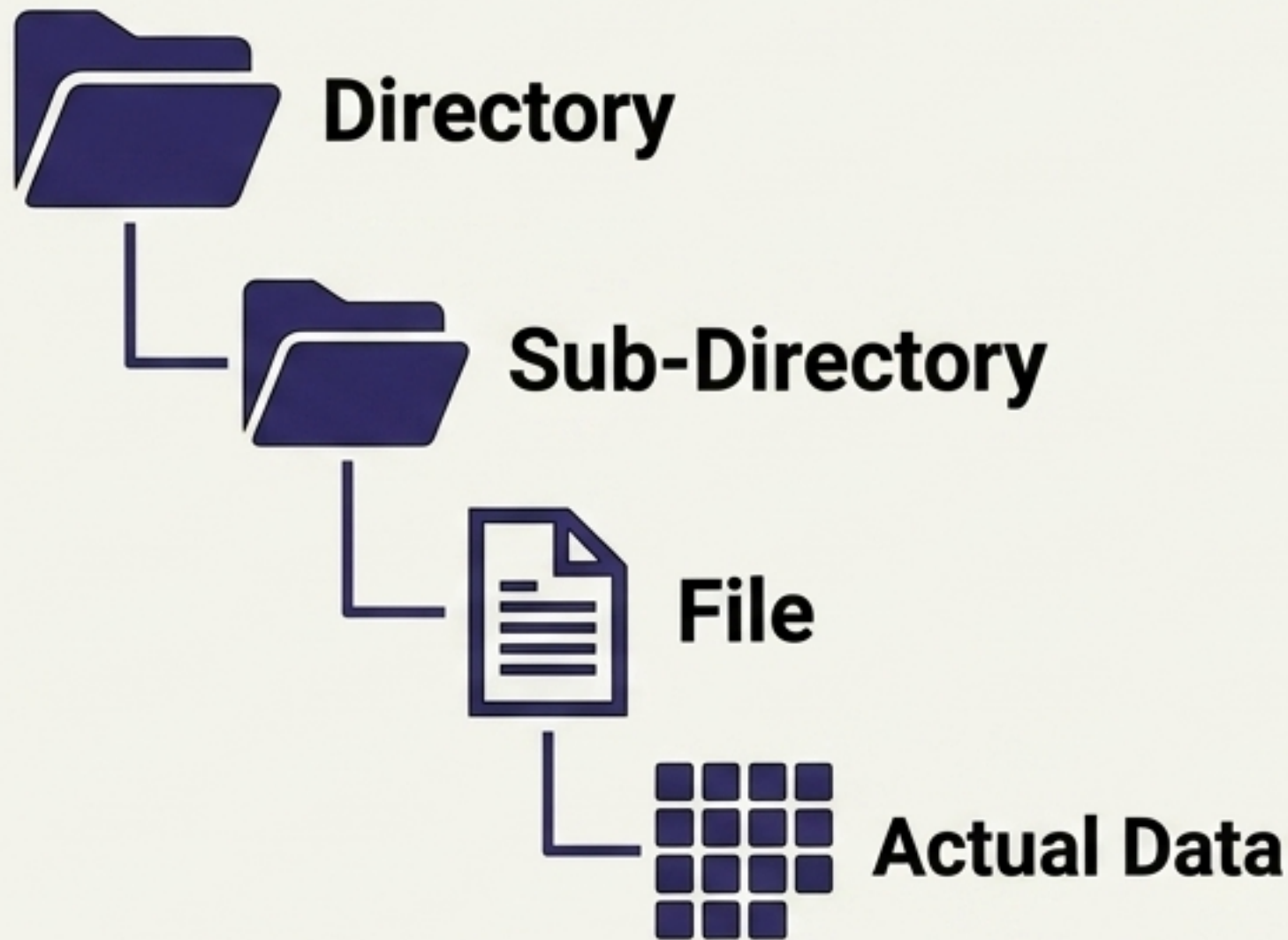
The local Operating System and HDFS remain strictly isolated

Hadoop runs on top of a host OS (like Linux), but maintains a completely separate storage environment.



Key Rule: Deleting a file on the local Linux desktop has zero impact on HDFS. Deleting a file in HDFS has zero impact on the Linux host. Data only crosses the boundary via explicit transfer commands.

Data storage syntax demands a rigid organizational hierarchy



1. **Directory** (e.g., /test1)
-> Must be created first.

2. **Sub-Directory** (e.g., /data1)
-> Nested within the parent.

3. **File** (e.g., file.txt)
-> The container for the data.

4. **Actual Data**
-> Resides exclusively within the file.

Base Command Syntax: All operations begin with the distributed file system prefix: `bin/hdfs dfs -[command]`

Operations Cheat Sheet:

Directories & Files

Create a Directory:	-mkdir /[directory_name]
List Contents (Current):	-ls /[path]
List Contents (Recursive):	-lsr /[path] (Shows all nested files/folders)
Remove a File:	-rm /[file_path]
Remove a Directory:	-rm -r /[directory_path]
Change Permissions:	-chmod 755 /[path] (Updates read/write/execute access)

Note: All commands are prefixed with bin/hdfs dfs

Operations Cheat Sheet: Data Transfer & Movement

Linux (Local System)

HDFS (Distributed Cluster)

Push to Cluster (Linux -> HDFS):
-put [local_source] [hdfs_dest]
(Alternative: **-copyFromLocal**)



Pull to Local (HDFS -> Linux):
-get [hdfs_source] [local_dest]
(Alternative: **-copyToLocal**)



Internal HDFS Copy:
-cp [source_path] [dest_path]
(Duplicates data within cluster)



Internal HDFS Move:
-mv [source_path] [dest_path]
(Cuts and pastes data within cluster)

Operations Cheat Sheet: Data Inspection & Space Management

Read Entire File:

-cat /[file_path]
(Outputs full content to terminal)

Read Partial File:

-head (Reads top records) | **-tail**
(Reads bottom records)

Check Directory Size:

-du /[path] (Returns size in bytes)

Check Global Drive Size:

-df (Returns cluster-wide space usage)

Detailed Metadata Count:

-count /[path] (Returns sub-directory count, file count, and total size)

Verify File Existence:

-test -t /[file_path]

Note: All commands are prefixed with bin/hdfs dfs

The Complete Big Data Pipeline

From local machine ingestion to distributed block storage, YARN resource allocation, and parallel MapReduce processing—Hadoop remains the foundational blueprint powering the modern data warehouse and advanced cloud analytics.

